

VIDEO NOTES · PART 5

Working With LLM Providers

Integrations, local models, and tuning model behaviour

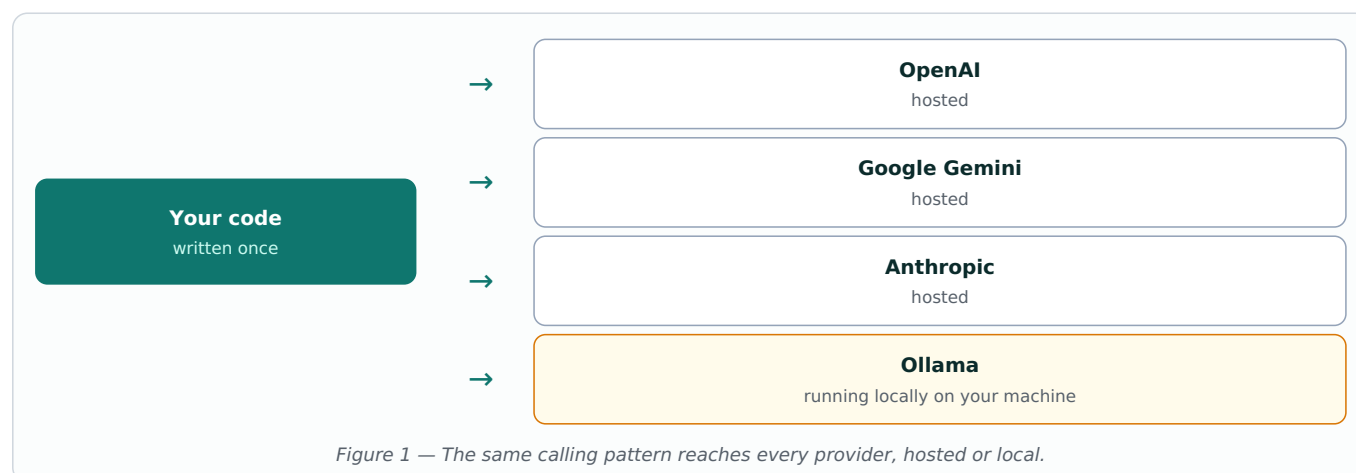
What this document is

A plain-English guide to calling different model providers through LangChain, and to the parameters that control how a model responds.

1 One Interface, Many Models

This lecture puts into practice the promise made earlier in the course: your application code should not care which model provider is behind it.

Four options are covered — three hosted services and one that runs on your own machine.

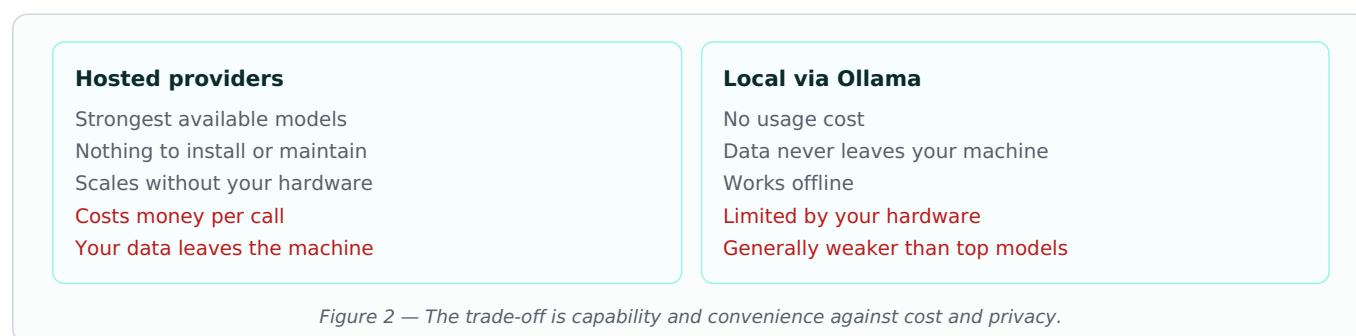


Setting up each hosted provider follows the same three steps: install its integration package, supply an API key, and create the chat model object. After that, everything you do with it is identical.

Why the course spends time on this: providers change. Prices shift, a new model outperforms the one you started on, or a client requires their data stay on their own infrastructure. Being able to move is worth more than picking correctly the first time.

2 Running Models Locally With Ollama

Ollama lets you download an open model and run it on your own computer. There is no API key and no network call — the model is simply running on your machine.

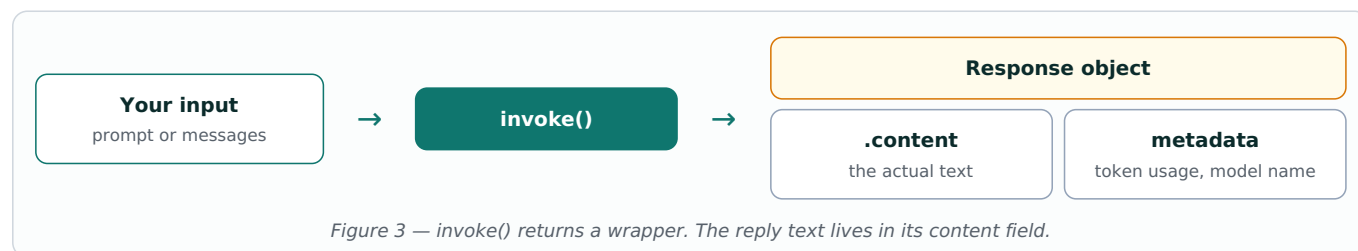


A practical development habit: build and debug against a local model where every run is free, then switch to a hosted provider through your config file when you need full quality. Because the interface is the same, nothing else changes.

3 Understanding invoke()

invoke() is the method that actually runs a model. You give it your input, it returns a response.

The important detail is that it does not hand back a plain string. It returns a response **object** — the text you want sits inside it, alongside extra information about the call.



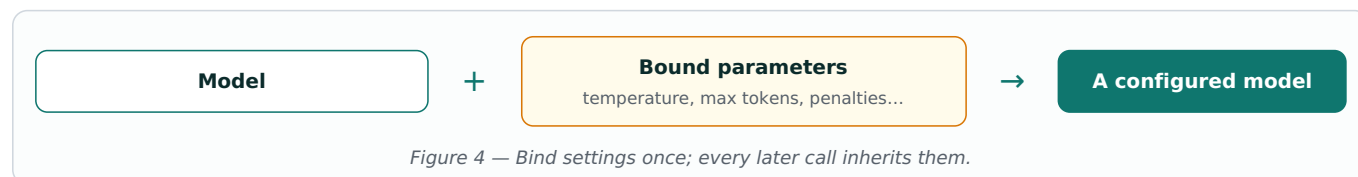
The beginner mistake: printing the response object directly and seeing a messy block of internal fields. Reach for the content field to get the reply itself.

The metadata is not clutter — it usually reports how many tokens the call consumed, which is how you track spend and diagnose why a response was cut short.

4 Model Parameters

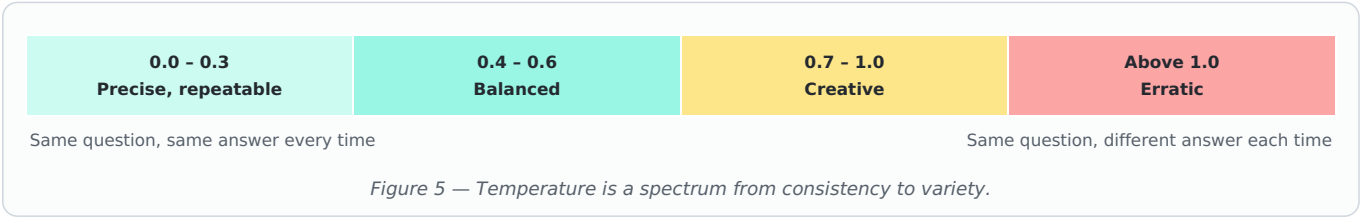
The largest part of this lecture covers the settings that shape *how* a model responds. The prompt decides what you ask; the parameters decide the character of the answer.

These settings are **bound** to the model — configured once when you create it, then applied to every call it makes.



Temperature — how predictable the answer is

Temperature controls randomness. At a low value the model consistently picks the most likely next word; at a high value it is willing to choose less likely ones.



Task	Sensible temperature
Classification, extraction, data mapping	Very low — you want the same answer every run
Factual question answering	Low
Summaries and rewriting	Low to moderate
Brainstorming, marketing copy, story ideas	Higher — variety is the point

A useful test: if two people running the same input should get the same output, your temperature is too high.

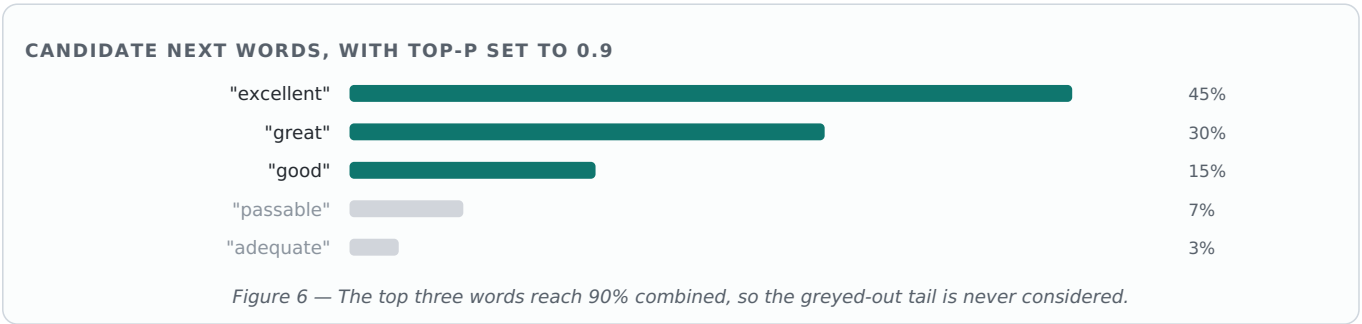
Max Tokens — a hard ceiling on length

This caps how much the model is allowed to generate. A token is roughly three quarters of a word in English.

The gotcha worth remembering: max tokens does not ask for a shorter answer. It cuts generation off at the limit — which can leave a sentence unfinished mid-word. To genuinely get a shorter reply, say so in the prompt. Use max tokens as a safety limit on cost, not as a style instruction.

Top-P — narrowing the pool of candidate words

At every step the model has many possible next words, each with a probability. Top-P (nucleus sampling) keeps only the most likely ones, up to a combined probability you specify, and ignores the rest.



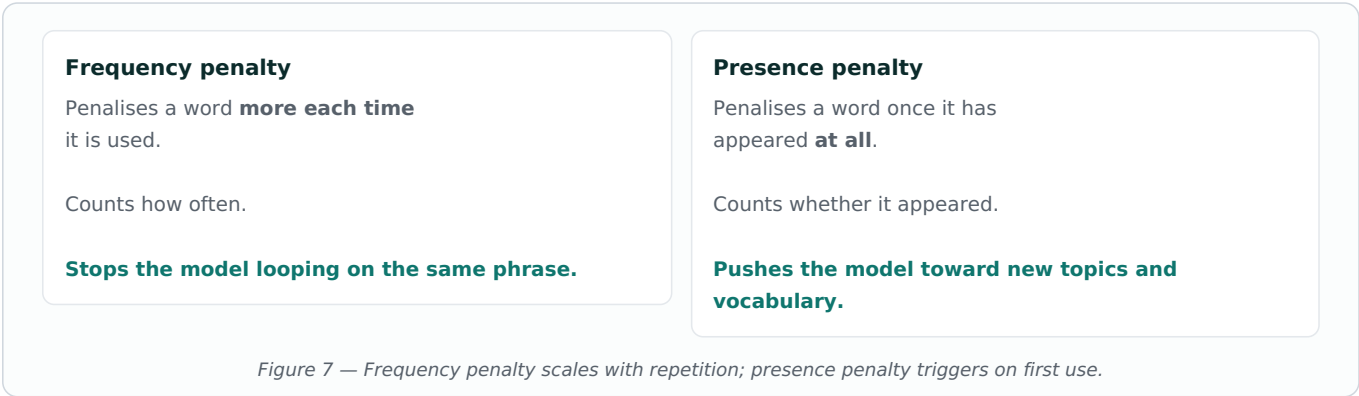
A low top-p keeps the model tightly on the obvious choices. A high value lets unusual words back into play.

Temperature	Top-P
Changes how boldly the model chooses among the available words	Changes which words are available to choose from at all

Standard advice: tune one of these, not both. Adjusting them together makes results hard to reason about, because you can no longer tell which setting caused a change. Most teams leave top-p at its default and use temperature.

Frequency and Presence Penalties

Both discourage repetition, but they work differently, and the distinction is the thing people most often get wrong.

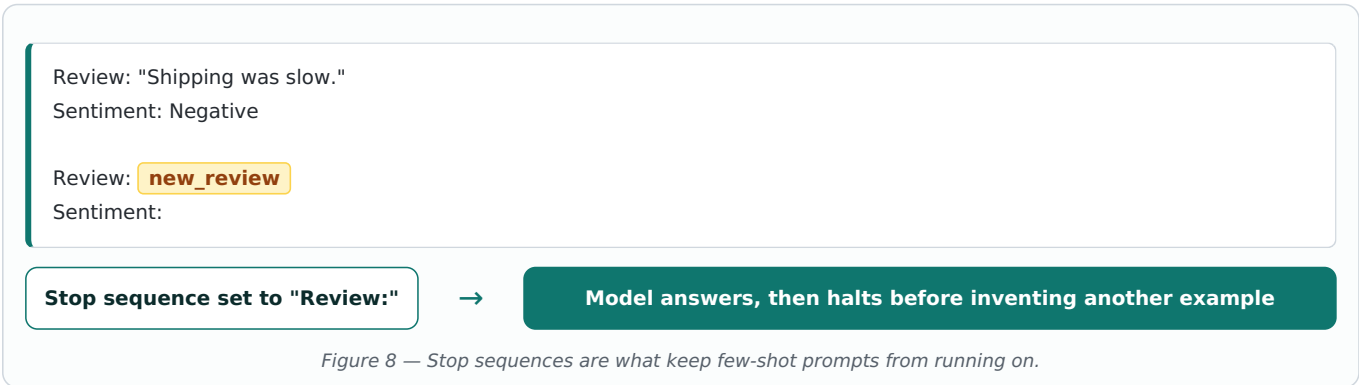


Symptom	Which one to reach for
The same sentence or phrase keeps reappearing	Frequency penalty
The output is on topic but circles the same few ideas	Presence penalty
Writing feels flat and repetitive in vocabulary	A little of both

Do not overdo them. Pushed too high, the model starts avoiding words it genuinely needs — including key terms from your own prompt — and the output becomes strained. Small values are usually enough.

Stop Sequences — where generation ends

A stop sequence is a piece of text that, when the model produces it, halts generation immediately. The stop text itself is not included in the output.



This pairs directly with the few-shot prompting from the previous lecture. Given a pattern, a model will happily continue it forever — generating fake reviews and grading them too. A stop sequence ends the reply at the right point.

5 Parameter Cheat Sheet

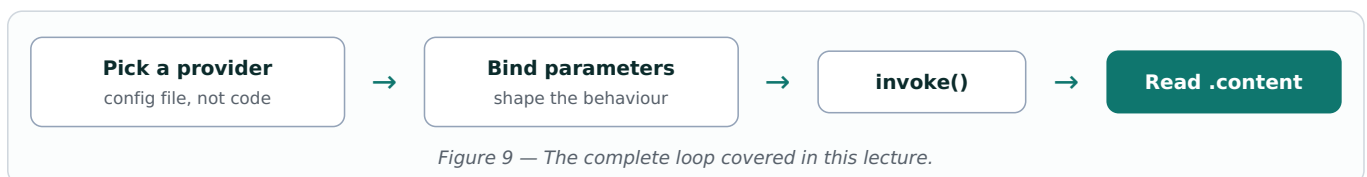
Parameter	What it controls	Reach for it when
Temperature	Randomness of word choice	Output is too repetitive, or too unpredictable
Max tokens	Hard ceiling on output length	You need to cap cost or guard against runaway replies
Top-P	Which candidate words are eligible	You want tighter word choice without touching temperature
Frequency penalty	Discourages reusing the same words	Phrases keep looping
Presence penalty	Discourages returning to used words	The answer keeps circling one idea
Stop sequences	Where generation must halt	The model runs past the point you wanted

Starting points by task type

Building this	Sensible starting configuration
Classifier or extractor	Temperature near zero, a modest max tokens, and a stop sequence if you are using few-shot examples. Consistency matters more than anything else.
Support chatbot	Low to moderate temperature, a max token cap so replies stay short, and a small presence penalty to avoid repeating the same reassurance.
Creative writing helper	Higher temperature, a generous max tokens, and modest frequency and presence penalties for varied language.
Summariser	Low temperature for faithfulness, with the target length stated in the prompt rather than enforced by max tokens.

How to tune in practice: change one parameter at a time and keep the prompt fixed. If you change two at once you learn nothing about either.

6 Recap



Practice before the next lecture

- ☐ Run the same prompt through two different providers and compare the replies.
- ☐ Install Ollama, pull one small model, and run an inference with no API key.
- ☐ Print a full response object once, then print only its content field.
- ☐ Run one prompt at temperature 0 three times, then at 1.0 three times.
- ☐ Set max tokens deliberately low and observe the reply being cut off mid-sentence.
- ☐ Add a stop sequence to a few-shot prompt and watch the runaway output stop.

The short version: LangChain makes the provider a configuration choice rather than a code dependency; `invoke()` returns an object whose `content` field holds the text; and the parameters bound to a model are what turn a generic assistant into one suited to your specific task.